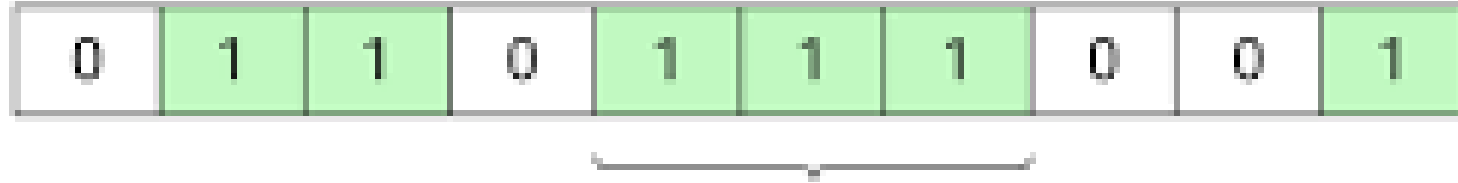




Longest Sequence of 1 bit

Write a program that takes a Integer as input and counts the length of the longest sequence of consecutive 1's in that binary number. Can you provide an example of a binary number and its corresponding longest sequence of 1's?



1. The `LongestSequenceOfOnes` class is defined to contain the code for finding the longest sequence of 1s in a binary number.
2. The `longestSequenceOfOnes` method takes an integer `binaryNumber` as input and returns an integer representing the length of the longest sequence of 1s in its binary representation.
3. Inside the `longestSequenceOfOnes` method:
 - The `binaryNumber` is converted to a binary string using the `Integer.toBinaryString` method and stored in the `binaryString` variable.
 - Two variables, `count` and `maxCount`, are initialized to 0. These variables will keep track of the current sequence count and the maximum sequence count encountered so far, respectively.
4. A loop is used to iterate over each character in the `binaryString`:
 - The loop variable `digit` represents the current character being processed.
 - If `digit` is equal to '1', indicating a 1 in the binary sequence:
 - The `count` variable is incremented to keep track of the current sequence of consecutive 1s.
 - If the current `count` exceeds the `maxCount`, it means a new maximum sequence has been found, so `maxCount` is updated accordingly.
 - If `digit` is not equal to '1', indicating the end of a sequence:
 - The `count` variable is reset to 0 to start counting a new sequence.

5. After the loop, the ``maxCount`` variable holds the length of the longest sequence of 1s in the binary number.

6. The method returns the ``maxCount`` as the result.

7. The ``main`` method serves as the entry point of the program.

- An example binary number, ``0b1101110111111001`` (which is equivalent to the decimal number 87769), is stored in the ``binaryNumber`` variable.

- The ``longestSequenceOfOnes`` method is called with ``binaryNumber`` as the argument, and the result is stored in the ``longestSequence`` variable.

- Finally, the result is printed to the console using ``System.out.println``.

When you run the code, it calculates the longest sequence of 1s in the binary number ``0b1101110111111001`` and prints the result as:

```
...
```

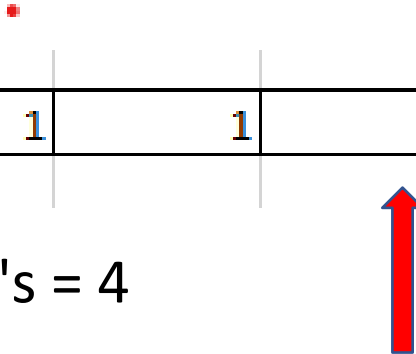
```
The longest sequence of 1's in 1101110111111001 is: 5
```

```
...
```

This means that the binary number ``0b1101110111111001`` contains a sequence of 1s with a length of 5, which is the longest sequence in the number.

```
1. public class LongestSequenceOfOnes {
2.     public static int longestSequenceOfOnes(int binaryNumber) {
3.         String binaryString = Integer.toBinaryString(binaryNumber);
4.         int count = 0;
5.         int maxCount = 0;
6.
7.         for (char digit : binaryString.toCharArray()) {
8.             if (digit == '1') {
9.                 count++;
10.                if (count > maxCount) {
11.                    maxCount = count;
12.                }
13.            } else {
14.                count = 0;
15.            }
16.        }
17.        return maxCount;
18.    }
19.    public static void main(String[] args) {
20.        int binaryNumber = 0b1101110111111001; // Binary representation of 87769
21.        int longestSequence = longestSequenceOfOnes(binaryNumber);
22.        System.out.println("The longest sequence of 1's in " +
23.            Integer.toBinaryString(binaryNumber) + " is: " + longestSequence);
24.    }
25.}
```

After Flipping one '0' to '1'



1	0	0	1	1	0	1	1	1	1	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---

Before Flipping count 1's = 4

1	0	0	1	1	1	1	1	1	1	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---

After Flipping count 1's = 7

1. The ``Longafterflip`` class is defined to contain the code for finding the longest consecutive sequence of 1s after flipping a single bit in a binary number.
2. The ``longestConsecutiveOnes`` method takes an integer ``n`` as input and returns an integer representing the length of the longest consecutive sequence of 1s.
3. Inside the ``longestConsecutiveOnes`` method:
 - The input number ``n`` is converted to a binary string using ``Integer.toBinaryString`` and stored in the ``binary`` variable.
 - Three variables are initialized: ``maxLength`` to store the maximum length, ``currentLength`` to keep track of the current length of consecutive ones, and ``previousLength`` to store the length of the previous consecutive sequence of ones encountered.
4. A loop is used to iterate over each bit in the ``binary`` string:
 - The loop variable ``bit`` represents the current bit being processed.
 - If ``bit`` is equal to `'1'`, indicating a consecutive sequence of ones:
 - The ``currentLength`` variable is incremented.
 - If ``bit`` is not equal to `'1'`, indicating a bit `'0'`:
 - The ``maxLength`` is updated by taking the maximum of the current length, previous length, and adding 1 (representing flipping the current `'0'` bit to `'1'`).
 - The ``previousLength`` is updated to the current length.
 - The ``currentLength`` is reset to 0 to start counting a new sequence.

5. After the loop ends, the ``maxLength`` is updated one more time, considering the final consecutive sequence of ones encountered.
6. The method returns the ``maxLength`` as the result.
7. The ``main`` method serves as the entry point of the program.
 - An example binary number, ``0b111011110``, is stored in the variable ``n``.
 - The ```

```
1. public class Longafterflip {
2.     public static int longestConsecutiveOnes(int n) {
3.         String binary = Integer.toBinaryString(n);
4.         int maxLength = 0;
5.         int currentLength = 0;
6.         int previousLength = 0;
7.         for (char bit : binary.toCharArray()) {
8.             if (bit == '1') {
9.                 currentLength++;
10.            } else {
11.                maxLength = Math.max(maxLength, currentLength + previousLength + 1);
12.                previousLength = currentLength;
13.                currentLength = 0;
14.            }
15.        }
16.        maxLength = Math.max(maxLength, currentLength + previousLength + 1);
17.        return maxLength;
18.    }
19.    public static void main(String[] args) {
20.        int n = 0b111011110;
21.        System.out.println(longestConsecutiveOnes(n)); // Output: 8
22.    }
```



THANK YOU